

NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY



COMPUTER HARDWARE SOFTWARE WORKSHOP

COCSC19

IOT BASED DUAL AXIS SOLAR TRACKER AND WEATHER MONITORING

Submitted by:

VIKASH KUMAR (2022UCS1625)

GAURAV GAHLOT (2022UCS1647)

AKSHAJ GUPTA (2022UCS1667)

1. PROBLEM STATEMENT

The looming energy crisis, compounded by the depletion of conventional resources, has highlighted the urgent necessity for transitioning towards sustainable energy solutions. Traditional sources like coal and petroleum, once instrumental in driving industrialization, now face imminent scarcity and environmental degradation, exacerbating pollution and contributing significantly to global warming. Amidst this backdrop, the imperative to adopt cleaner and more abundant energy alternatives has become increasingly apparent.

Solar energy emerges as a beacon of hope in this quest for sustainability, characterized by its inexhaustible nature and minimal environmental footprint. Despite extensive efforts to harness solar energy efficiently, challenges persist in optimizing radiation collection efficiency. One promising avenue in this pursuit is the utilization of dual-axis solar trackers, offering a highly efficient means of maximizing solar energy utilization. These trackers dynamically adjust the orientation of solar panels to align with the sun's trajectory, holding immense potential in enhancing energy capture efficiency.

2. INTRODUCTION

Energy demands in the recent years have been recorded to be growing at an exponential rate by the commercial and as well as domestic markets. While the non-renewable resources are rapidly getting depleted, it leaves no other option but to use renewable resources to produce usable energy. One of these resources, the Solar energy is the most abundant and easy to harness resource through Solar panels. This project makes this process of harnessing solar energy more efficient. So here we propose an automated IOT based solar power monitoring system that allows for automated solar power monitoring from anywhere over the internet. We use arduino based system to monitor solar panel parameters. Our system constantly monitors the solar panel and transmits the power output to IOT system over the internet. The Rotating Solar Panel system scans from one horizon to other to know the current position of sun and hence the position from which the greater solar energy can be harnessed. The position which has the highest energy capacity is chosen to charge the Battery. In this way we can harness the most of from the Solar panel by adjusting it to be incident directly towards Sun consistently. Thus this project makes this process of harnessing solar energy more efficient and hence smarter.

3. OBJECTIVE

This tracker aims to orient solar panels toward the sun throughout the day to maximize energy capture.

- **Predictive tracking** :- Collect data over time about the sun's position, weather conditions, and energy output from the solar panel.
- **Cloud Cover** :- Integrate a weather sensor or use an online weather API to get real-time data about cloud cover.
- **Energy yield optimization** :- Collect data on energy production from the solar panel and environmental factors.
- **Anomaly Detection** :- anomaly detection can be used to identify malfunctions or unusual conditions that may affect the solar panel's performance.

4. Apparatus Required

- arduino (300-500)
- servo motor (150)
- solar panel (100-500)
- solar panel mount(300-500)
- LCD display (200)
- resistor-transistor-capacitor-led-push button-switch(500)
- wires(100)
- PCB and breadboard (200)
- IC(200)
- temp sensor(50)
- ldr (25)
- current and voltage(200)

5. HARDWARE

- The system begins with solar panels harvesting energy from sunlight, which is then stored in a battery. This battery serves as the power source for the entire setup. Four Light Dependent Resistor (LDR) sensors, connected to an Arduino Nano via analog pins A0-A3, continuously monitor light intensity levels to assess sunlight conditions.
- Based on this data, the Arduino Nano provides input to a motor driver, which adjusts the orientation of the solar panels for optimal energy capture. The solar panels are mounted on a dual-axis solar tracker, allowing for precise adjustments in both horizontal and vertical axes to maximize energy capture efficiency throughout the day.
- The battery not only powers the motor driver and Arduino Nano but also operates additional components. These include a small fan and LED, which are powered by the battery, demonstrating practical energy usage.
- Furthermore, the battery supplies power to a NodeMCU, facilitating connectivity with environmental sensors such as the DHT11 and BMP180. The NodeMCU collects data on temperature, humidity, and atmospheric pressure, enhancing the system's environmental monitoring capabilities.

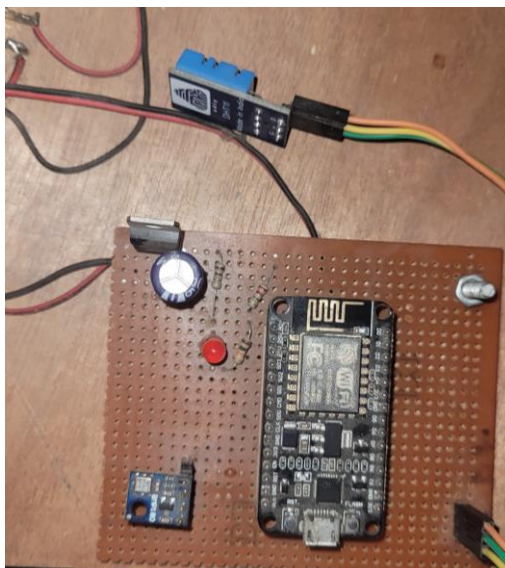
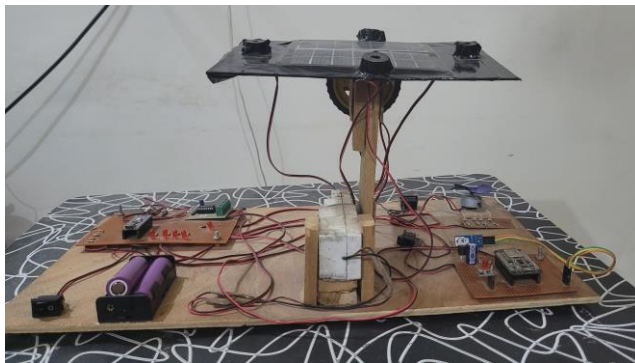
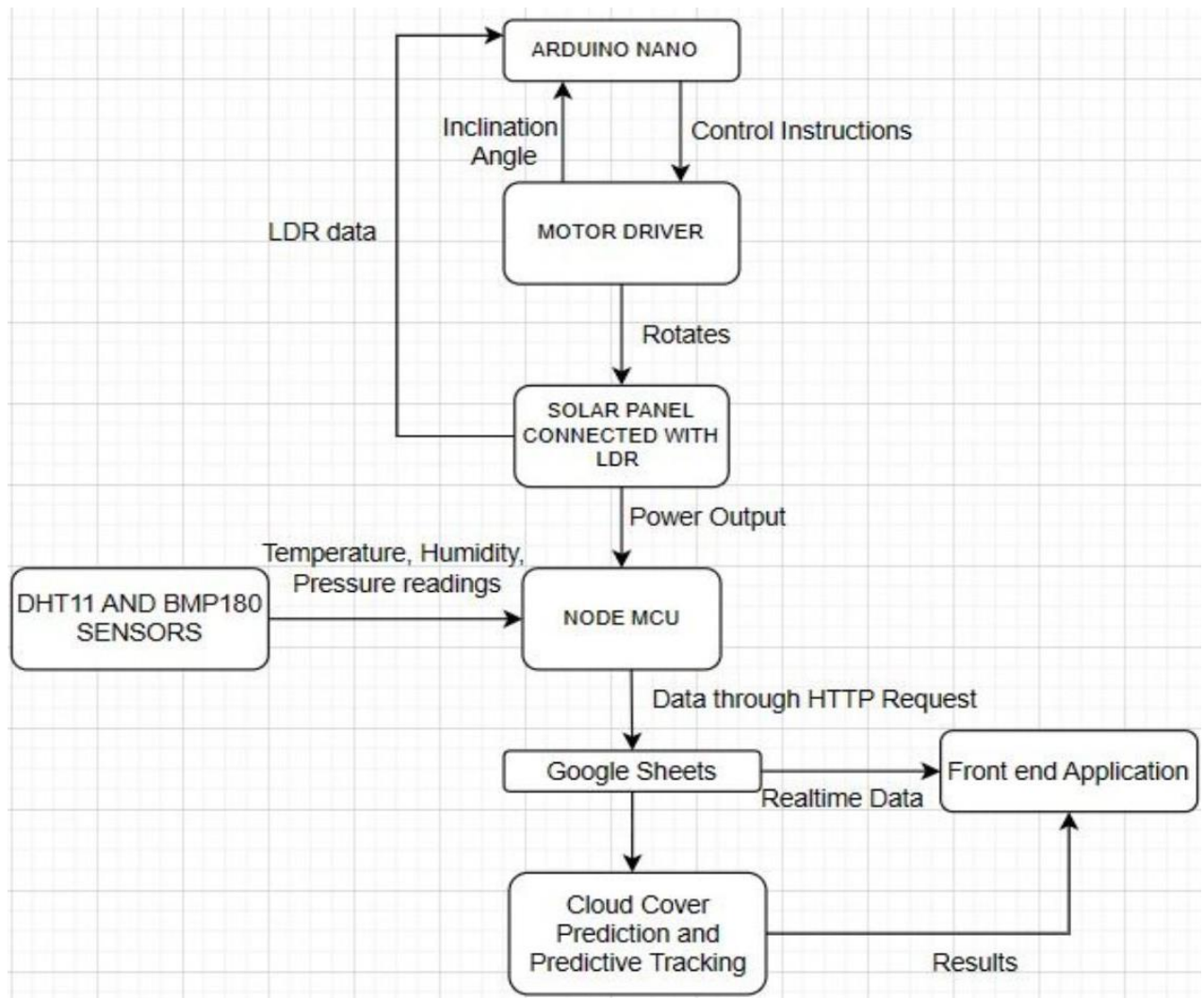


DIAGRAM:



6. SOFTWARE

6.1 IOT INTEGRATION:

Google Sheets Integration:

- Environmental data collected by NodeMCU and solar panel power output from Arduino Nano are securely transmitted to Google Sheets via HTTP request.
- Google Sheets serves as a centralized platform for storing, analyzing, and visualizing the collected data, enabling long-term monitoring and trend analysis.

6.2 MACHINE LEARNING FOR ANOMALY DETECTION:

- A dataset was created using real-time outputs from the system under normal environmental conditions.
- The dataset includes the following parameters:
Air Pressure, Humidity, Temperature, Altitude, and Solar Output.
- An XGBoost machine learning model was trained to predict solar output using the environmental features.
- This approach ensures reliable solar power generation.
- In real-world deployment, the system:
 - Predicts the expected solar output in real time.
 - Compares it with the actual output measured.
 - Detects anomalies based on the deviation between expected and actual values.
- This approach ensures reliable solar power generation.

6.3 MACHINE LEARNING MODEL FOR RAIN PREDICTION:

- A dataset sourced from Kaggle includes the following features:
Temperature, Humidity, Wind Speed, Cloud Cover, Pressure, and Rain status.
- Objective: Predict Rain / No Rain using environmental parameters.
- Data Sources:
 - DHT & BMP Sensors: Capture Temperature, Humidity, and Pressure.
 - API Calls: Retrieve Wind Speed and Cloud Cover in real time.
- An XGBoost classifier is trained to accurately predict the likelihood of rain.
- SMOTE analysis was applied to balance both the classes.
- This model enhances the system's ability to anticipate weather changes.

7. CODE

7.1 Code for anomaly detection

```
from xgboost import XGBRegressor
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import pandas as pd
```

```
file_path = 'weather data.csv' # Replace with your file path
data = pd.read_csv(file_path)
```

```
data['datetime'] = pd.to_datetime(data['date'] + ' ' + data['time'])

# Drop unnecessary columns
data = data.drop(columns=['id', 'date', 'time'])
```

```
X = data[['temperature', 'humidity', 'surface_pressure', 'altitude']]
y = data['Solar Power (mili watts)']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
model = XGBRegressor(random_state=42)
model.fit(X_train_scaled, y_train)
```

[18]

...

XGBRegressor

XGBRegressor(base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=None, device=None, early_stopping_rounds=None, enable_categorical=False, eval_metric=None, feature_types=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=None, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=None, max_leaves=None, min_child_weight=None, missing=nan, monotone_constraints=None, multi_strategy=None, n_estimators=None, n_jobs=None, num_parallel_tree=None, random_state=42, ...)

```
y_pred = model.predict(X_test_scaled)
```

```
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
print(f"Model Accuracy (R2 Score): {r2:.2f}")
print(f"Mean Absolute Error: {mae:.2f}")
```

Result:

```
Model Accuracy (R2 Score): 0.91
Mean Absolute Error: 0.28
```


7.2 Code for rain prediction:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
import xgboost as xgb
from sklearn.metrics import accuracy_score, classification_report

# Load the dataset
data_path = "D:\College\sem 6\chsw\Major Project\weather_forecast_data.csv"
df = pd.read_csv(data_path)

# Display dataset info
print(df.info())

# Assuming 'rain' is the target column
target_column = "Rain"
if 'Rain' in df.columns:
    df['Rain'] = df['Rain'].map({'rain': 1, 'no rain': 0})

df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Temperature  2500 non-null   float64
1   Humidity     2500 non-null   float64
2   Wind_Speed   2500 non-null   float64
3   Cloud_Cover  2500 non-null   float64
4   Pressure     2500 non-null   float64
5   Rain         2500 non-null   object
dtypes: float64(5), object(1)
memory usage: 117.3+ KB
None
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	103237.87590	1
1	27.879734	46.489704	5.952484	4.990053	99261.41895	0
2	25.069084	83.072843	1.371992	14.855784	100723.16200	0
3	23.622080	74.367758	7.050551	67.255282	98263.20127	1
4	20.591370	96.858822	4.643921	47.676444	98082.51417	0

```

X = df.drop(columns=[target_column])
y = df[target_column]

# Apply SMOTE to handle class imbalance
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.2, random_state=42
)
print("Class distribution after SMOTE:\n", pd.Series(y_resampled).value_counts())

```

Class distribution after SMOTE:

```

Rain
1    2186
0    2186
Name: count, dtype: int64

```

```

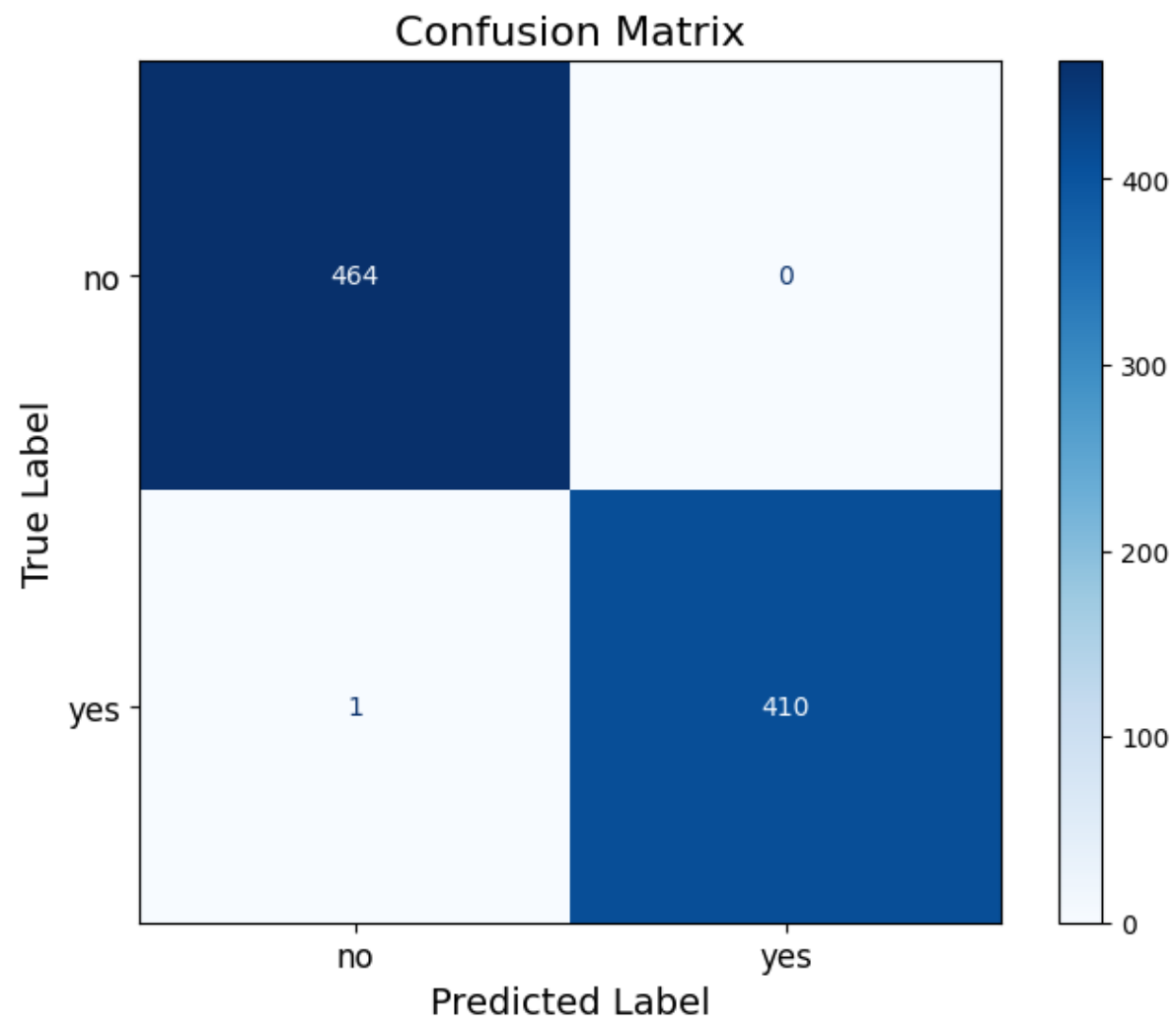
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
model.fit(X_train, y_train)
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Make predictions
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["no", "yes"])
fig, ax = plt.subplots(figsize=(8, 6)) # Adjust size as needed
disp.plot(ax=ax, cmap="Blues", values_format='d') # 'd' = integer format
plt.title("Confusion Matrix", fontsize=16)
plt.xlabel("Predicted Label", fontsize=14)
plt.ylabel("True Label", fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.grid(False)
plt.show()

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}")
print("Classification Report:\n", classification_report(y_test, y_pred))

```

Result



Model Accuracy: 0.9989					
Classification Report:					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	464	
1	1.00	1.00	1.00	411	
accuracy			1.00	875	
macro avg	1.00	1.00	1.00	875	
weighted avg	1.00	1.00	1.00	875	

7.3 ARDUINO CODE

```
arduino.ino
1  #include <Servo.h>
2
3  // Define the servo objects
4
5  Servo horizontalServo;
6
7  Servo verticalServo;
8
9  // Define the LDR sensor pins
10
11  int ldrPins[] = {A0, A1, A2, A3};
12  int numSensors = sizeof(ldrPins) / sizeof(ldrPins[0]);
13
14  // Define the servo angle limits
15
16  int horizontalMinAngle = 0;
17
18  int horizontalMaxAngle = 180;
19
20  int verticalMinAngle = 0;
21
22  int verticalMaxAngle = 90;
23
24  // Define the threshold for light intensity
25
26  int lightThreshold = 500;
27
28  void setup() {
29
30    // Attach servos to corresponding pins
31
32    horizontalServo.attach(9);
33
34    verticalServo.attach(10);
35    horizontalServo.write(horizontalMinAngle);
36
37    verticalServo.write(verticalMinAngle);
38
39    // Set LDR sensor pins as INPUT
40
41    for (int i = 0; i < numSensors; i++) {
42
43      pinMode(ldrPins[i], INPUT);
44
45    }
46
47  }
```

```
49 void loop() {
50
51   // Read light intensity from LDR sensors
52
53   int sensorValues[numSensors];
54
55   for (int i = 0; i < numSensors; i++) {
56
57     sensorValues[i] = analogRead(ldrPins[i]);
58
59   }
60   int avgIntensity = 0;
61
62   for (int i = 0; i < numSensors; i++) {
63
64     avgIntensity += sensorValues[i];
65
66   }
67
68   avgIntensity /= numSensors;
69
```

```
70 // Check if light intensity is above threshold
71
72 if (avgIntensity > lightThreshold) {
73
74   // Calculate servo angles based on sensor readings
75
76   int horizontalAngle = map(sensorValues[1] - sensorValues[0], 0, 1023, horizontalMinAngle, horizontalMaxAngle);
77
78   int verticalAngle = map(sensorValues[3] - sensorValues[2], 0, 1023, verticalMinAngle, verticalMaxAngle);
79
80   // Set servo angles to track the light source
81
82   horizontalServo.write(horizontalAngle);
83
84   verticalServo.write(verticalAngle);
85
86 }
87 // Delay before next iteration
88
89 delay(100);
90
91 }
```

7.4 NODEMCU CODE

```
1  #include <SPI.h>
2  #include <Wire.h>
3  #include <ESP8266WiFi.h>
4  #include <WiFiClientSecure.h>
5  #include <Adafruit_BMP085.h>
6  #include <DHT.h>
7
8  Adafruit_BMP085 bmp;
9
10
11 #define DHTPIN D4           // Pin connected to the DHT sensor
12 #define DHTTYPE DHT11      // DHT sensor type (DHT11 or DHT22)
13
14 DHT dht(DHTPIN, DHTTYPE);
15
16 const char* ssid = "Bruh";
17 const char* password = "Akki@453";
18
19 const char* host = "script.google.com"; //https://script.google.com/
20 const int httpsPort = 443;
21
22 WiFiClientSecure client;
23 String Gas = "AKfycbzGKIz74GCG4Y8Fw7teRPyVBBwIlp46jUMcRDBMQyLf9HFMTwhzoHIh1EBrJ9ASek";
24
25
26 const int solarPanelPin = A0;
27
```

```
28 void setup() {
29     Serial.begin(9600);
30     WiFi.begin(ssid, password);
31     Serial.print("Connecting");
32     while (WiFi.status() != WL_CONNECTED) {
33         Serial.print(".");
34         delay(250);
35     }
36
37     Serial.println("");
38     Serial.print("Successfully connected to : ");
39     Serial.println(ssid);
40     Serial.print("IP address: ");
41     Serial.println(WiFi.localIP());
42     Serial.println();
43     client.setInsecure();
44     dht.begin();
45     // if (!bmp.begin()) {
46     // Serial.println("Could not find a valid BMP085 sensor, check wiring!");
47     // while (1) {}
48     // }
49     bmp.begin();
50
51 }
```

```

53 void loop() {
54     float temperature = dht.readTemperature(); // Read temperature in Celsius
55     float humidity = dht.readHumidity();
56     int sensorValue = analogRead(solarPanelPin);
57     float voltage = sensorValue * (9 / 1023.0);
58     float power = voltage * 0.82;
59     Serial.print("Pressure = ");
60     Serial.print(bmp.readPressure());
61     Serial.println(" Pa");
62     Serial.print("Power:- ");
63     Serial.println(power);
64     Serial.print("Altitude:- ");
65     Serial.print(bmp.readAltitude());
66     Serial.println(" meters");
67     Serial.print("Temperature= ");
68     Serial.println(temperature);
69     if (!client.connect(host, httpsPort)) {
70         Serial.println("connection failed");
71         return;
72     }
73     String url = "/macros/s/" + Gas + "/exec?temperature=" + temperature + "&humidity=" + humidity;
74     client.print(String("GET ") + url + " HTTP/1.1\r\n" + "Host: " + host + "\r\n" + "User-Agent: " + userAgent);
75     delay(300000);
76     //delay(1000);
77 }
78

```

8. WEB APPLICATION

Solar Tracker Dashboard

Manual Prediction

Automated Prediction

Solar Power Output

Manual Rainfall Prediction

Enter the environmental conditions below:

Temperature (°C)

23.70

- +

Humidity (%)

89.60

- +

Wind Speed (m/s)

7.33

- +

Cloud Cover (%)

50.50

- +

Pressure (Pa)

103237.00

- +

Predict Rainfall

Predicted Rainfall: Yes

Dej

Automated Rainfall Prediction

Fetch & Predict

	Timestamp	Temperature	Humidity	Pressure	Altitude	Solar Power
1	3/29/2025 15:03:27	29.1	36.7	98,827	209.83	0.11
2	4/14/2025 15:44:00	39.2	34.7	98,608	228.45	0.1
3	4/14/2025 15:49:01	33.7	41.7	97,781	299.07	7.39
4	4/14/2025 15:54:03	42.2	35.2	97,761	301.47	7.39
5	4/14/2025 16:02:19	49.5	31.5	97,594	315.8	7.39

Latest Prediction: No Rain

⚡ Solar Power Output Dashboard

☀ Predicted Output (milliWatts)

6.46 mW

☑ Actual Output (milliWatts)

7.39 mW

✅ System Operating Normally

📈 Solar Power Output Over Time

Solar Power Trend



9. CONCLUSION:

Successfully developed an IoT-based solar power monitoring and optimization system that integrates advanced sensors, real-time data analytics, and machine learning for intelligent energy management. The system incorporates dual-axis solar tracking to maximize energy capture by dynamically aligning solar panels with the sun's trajectory. Leveraging XGBoost models, it predicts solar output using environmental inputs and forecasts rain for weather-aware solar management. Anomaly detection ensures consistent operation by identifying unexpected performance drops. To enhance accessibility, I designed a Streamlit-based user interface that facilitates seamless interaction with models, delivers real-time predictions, and provides actionable system insights. This project exemplifies the powerful synergy of IoT, machine learning, and web-based interfaces in creating sustainable and intelligent energy solutions.